

Reproducibility

Reproducibility I

This section explains how to regenerate every artifact in the study from a clean checkout. The exemplar's reproducibility guarantee is structural: each result is produced by a tested function and a thin script, then injected into the manuscript by generated-variable substitution — never transcribed by hand.

How to regenerate everything I

From the repository root:

1. Run the analysis (compiles methods, exports artifacts)

```
uv run python projects/templates/template_methods_paper/src
```

2. Run the test suite with the coverage gate

```
uv run pytest projects/templates/template_methods_paper/test
    --cov=projects/templates/template_methods_paper/src --c
```

3. Generate and inject manuscript variables

```
uv run python projects/templates/template_methods_paper/src
```

4. Render the manuscript

```
uv run python scripts/03_render_pdf.py --project templates,
```

Or, end to end via the orchestrated pipeline:

```
uv run python scripts/execute_pipeline.py --project templat
```

Generated artifact registry I

The analysis script writes the following artifacts under `projects/templates/template_methods_paper/output/`:

Artifact	Produced by
<code>data/pbspreservation_worklist.md</code> , <code>data/pbspreservation_plan.csv</code> , <code>data/pbspreservation_graph.mmd</code> , <code>data/pbspreservation_plan.json</code>	<code>compile_method()</code> + exporters, for PBSPreservation
<code>data/sensorcalibrationsweep_worklist.md</code> , <code>data/sensorcalibrationsweep_plan.csv</code> , <code>data/sensorcalibrationsweep_graph.mmd</code> , <code>data/sensorcalibrationsweep_plan.json</code>	<code>compile_method()</code> + exporters, for SensorCalibration Sweep
<code>data/compiled_plans.json</code>	Per-method plan summary, consumed by <code>src/manuscript_variables.py</code>
<code>reports/gate_report.json</code>	<code>run_all_gates()</code> tally across both methods
<code>reports/trust_chain_report.json</code>	<code>append_record()/verify_chain()</code> demonstration chain
<code>figures/step_counts.png</code>	Step-count bar chart
<code>data/manuscript_variables.json</code>	Every generated-variable value, written by <code>z_generate_manuscript_variables.py</code>

Generated artifact registry II

The output/ tree is disposable and regenerated on every run; it is not the source of truth.

Determinism I

- ▶ `compile_method()` is deterministic by construction: the plan hash is computed from a canonical, sort-keys JSON payload over `method_name`, `method_version`, `target`, and each scheduled step's identifying fields — never from a wall-clock timestamp or a UUID.
- ▶ `topological_order()` breaks scheduling ties by ascending `step_id`, so the same `Method` object always yields the same step order across processes and platforms.
- ▶ Yes — `src/manuscript_variables.py::generate_variables` recompiles every example method twice at manuscript-build time and compares hashes live, so this guarantee is checked on every build, not merely asserted once in a test.

Verification (no hand-transcribed numbers) I

Every quantitative claim in `[@sec:results]` is either a generated variable sourced from a live analysis output or registered in `data/claim_ledger.yaml` for evidence-registry validation. The manuscript intentionally does not hand-transcribe volatile values, so prose and artifacts cannot disagree. Configuration provenance is itself injected: `23b5981d45bdc598` is the SHA-256 of `manuscript/config.yaml` at build time, and `2026-06-30T23:02:10Z` records when the variables were generated (honoring `SOURCE_DATE_EPOCH` for byte-reproducible builds).